

IA: MACHINE LEARNING FOUNDATIONS

Convolutional Neural Networks

Adrián Sánchez Morales

AGENDA

1. Introduction
2. Architecture
3. Building Blocks
4. Hyperparameters & Training
5. Other considerations
6. Applications



ie UNIVERSITY Driving Innovation

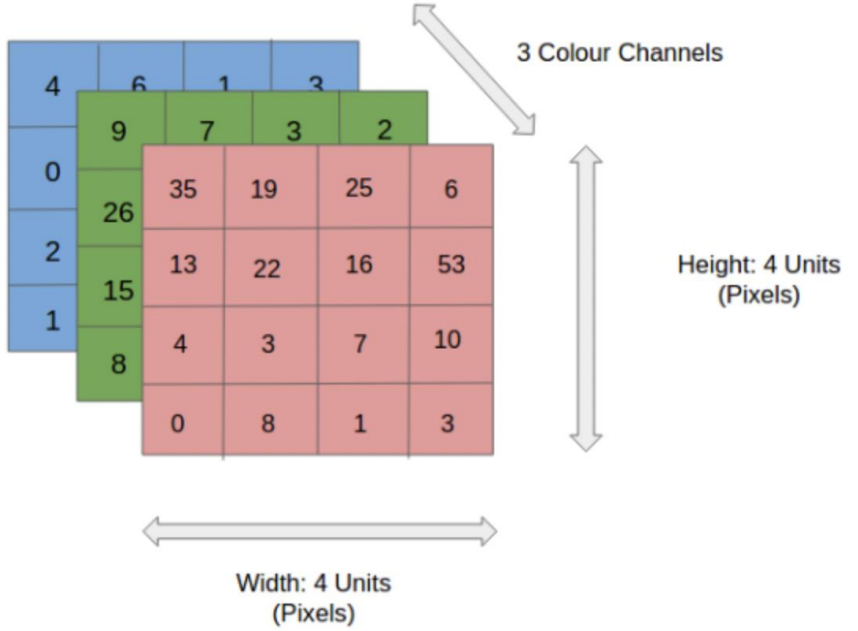
1. INTRODUCTION

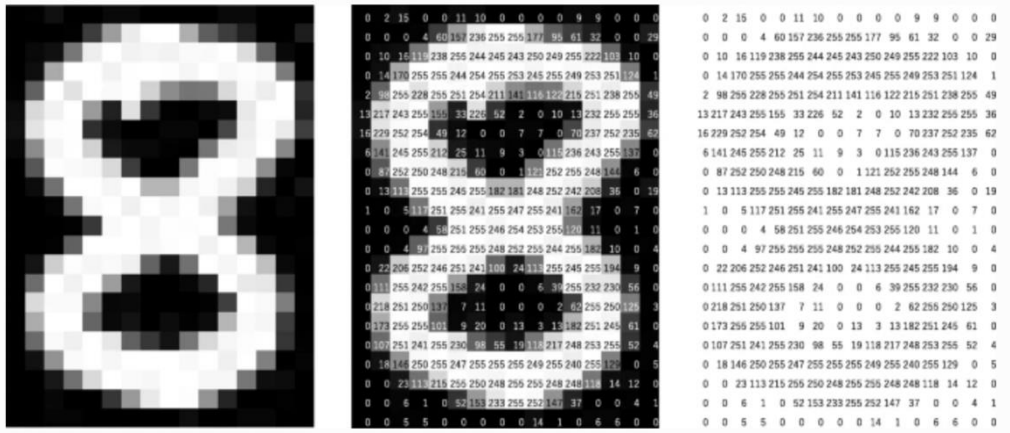
- To a computer, an image is just a **grid of numbers**. A CNN learns to find patterns in that grid (edges, textures, shapes) and uses them to recognize what the image shows.
- **Convolutional neural networks (CNNs)** are neural networks built around one operation, **convolution**: a small filter slides over the image and detects one pattern at a time.
- They are inspired by the **visual cortex**, where each neuron responds only to a small region of what the eye sees.
- Older methods needed filters designed by hand. A CNN **learns its filters from labelled examples**, so it needs little pre-processing.
- CNNs are behind photo search, face unlock, medical image screening, factory quality inspection and driver-assistance cameras.

1. INTRODUCTION

1.1. HOW A COMPUTER SEES AN IMAGE

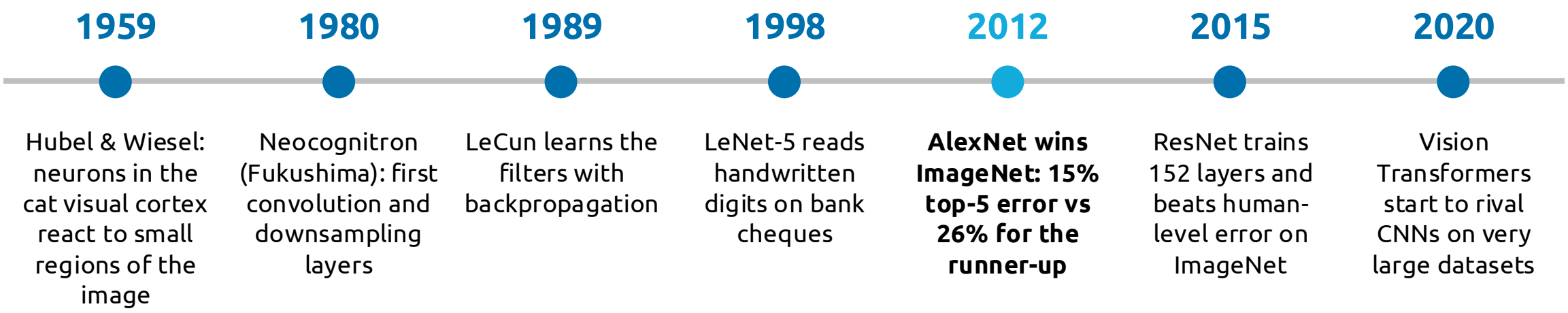
- A CNN is able to successfully capture the spatial patterns in an image through the application of relevant filters.
- An image can be mathematically represented as a *n*–dimensional matrix.
- In a color image, we have an RGB matrix that may be separated by its three-color planes — Red, Green, and Blue. There are a number of such color spaces in which images exist — Grayscale, RGB, HSV, CMYK, etc.
- A greyscale image is an array of numbers. The matrix on the right contains numbers between 0 and 255, each of which corresponds to the pixel brightness in the left image.





1. INTRODUCTION

1.2. HISTORY

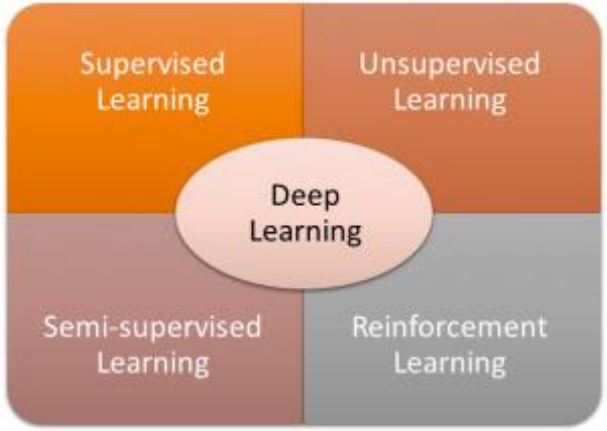


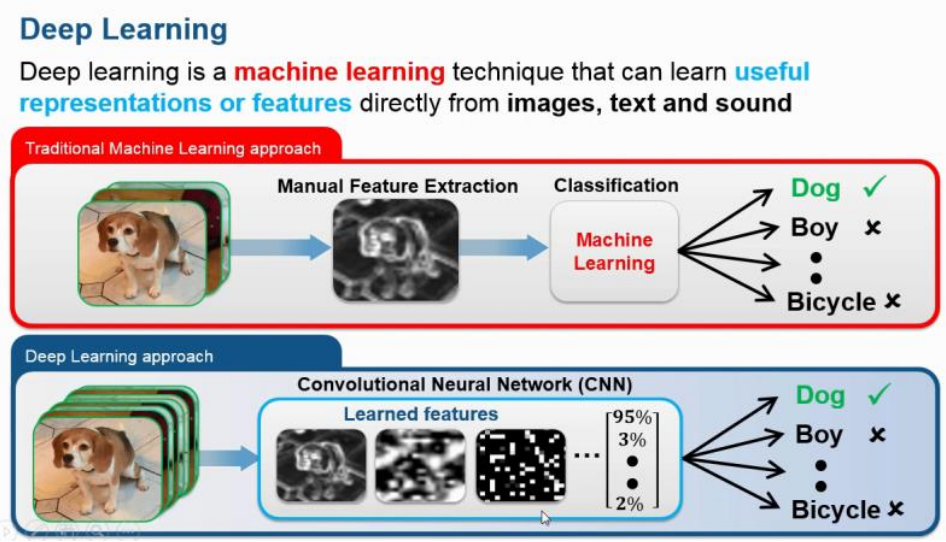
The ideas are from the 1980s; the 2012 breakthrough came from large labelled datasets (ImageNet) and GPUs.

1. INTRODUCTION

1.3. DEEP LEARNING

- Deep learning is a branch of machine learning based on a set of algorithms that learn to represent the data. Below we list the most popular ones.
    - Convolutional Neural Networks
    - Transformers (language and vision)
    - Autoencoders
    - Recurrent Neural Networks (RNN/LSTM/GRU)
    - Generative models (GANs, diffusion models)
- One of the promises of deep learning is that they replace hand-crafted feature extraction.
- The idea is that they will "learn" the best features needed to represent the given data.
- As a rule of thumb deeper models have the potential to perform better than shallow models. The problem is that the deeper you go the more data you will need to avoid over-fitting.





1. INTRODUCTION

1.3. DEEP LEARNING



AGENDA

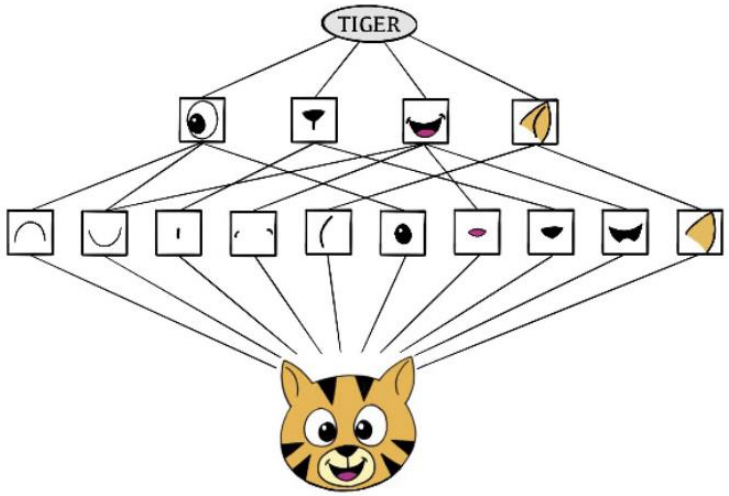
1. Introduction
2. Architecture
3. Building Blocks
4. Hyperparameters & Training
5. Other considerations
6. Applications



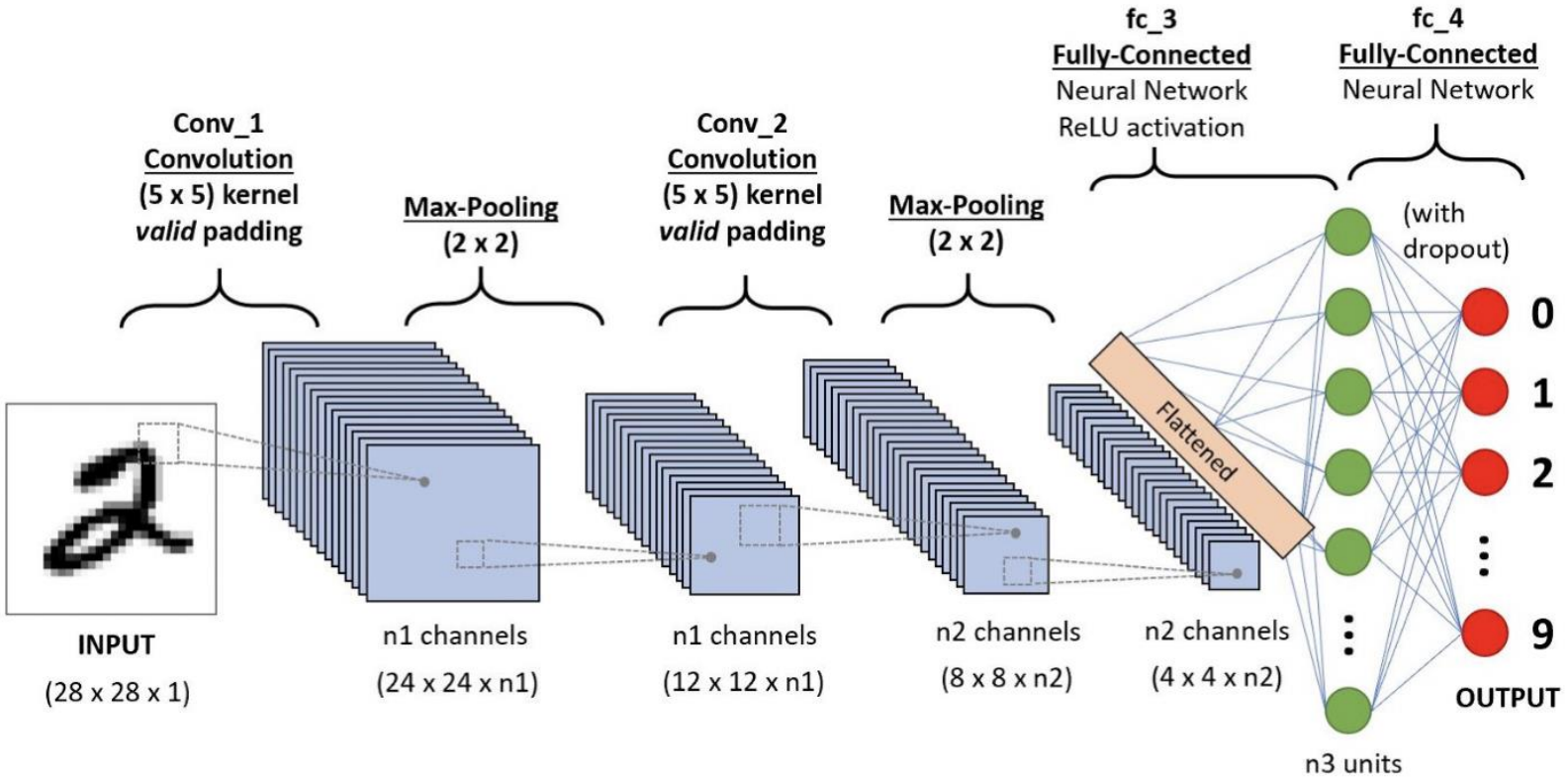
2. ARCHITECTURE

- A convolutional neural network consists of an input layer, hidden layers and an output layer.
- In a convolutional neural network, the hidden layers include layers that perform convolutions.
- In mathematics, convolution is a mathematical operation on two functions (f and g) that produces a third function that expresses how the shape of one is modified by the other.

- The network first identifies low-level features in the input image, such as small edges, patches of color, and the like.
- These low-level features are then combined to form higher-level features, such as parts of ears, eyes, and so on.
- Eventually, the presence or absence of these higher-level features contributes to the probability of any given output class.



2. ARCHITECTURE



2. ARCHITECTURE

2.1. ADVANTAGES OF CNNs OVER MLPs



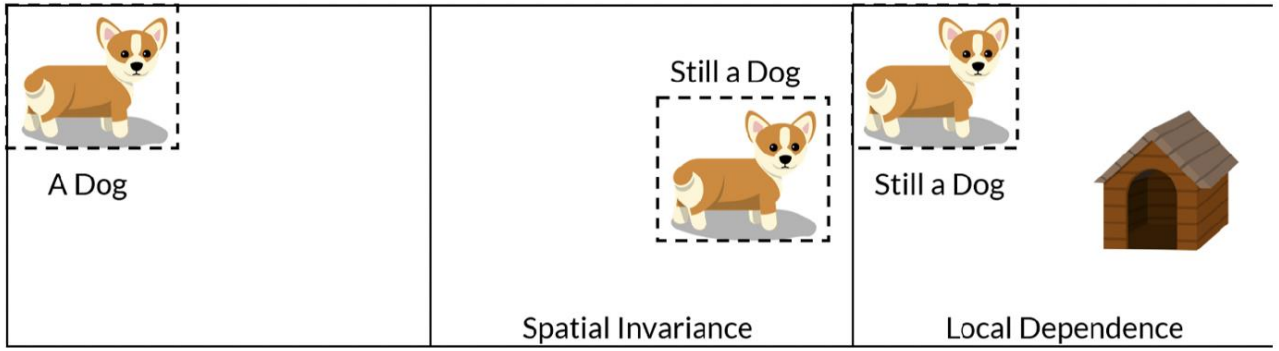
2. ARCHITECTURE

2.1. ADVANTAGES OF CNNs OVER MLPs

- MLPs use one perceptron for each pixel in an image and **the number of weights rapidly becomes unmanageable for large images**. Example: a 224×224 color image has 150,528 inputs, so a single fully connected neuron needs 150,528 weights. A 3×3 convolution filter needs only 27 weights and is reused across the whole image.

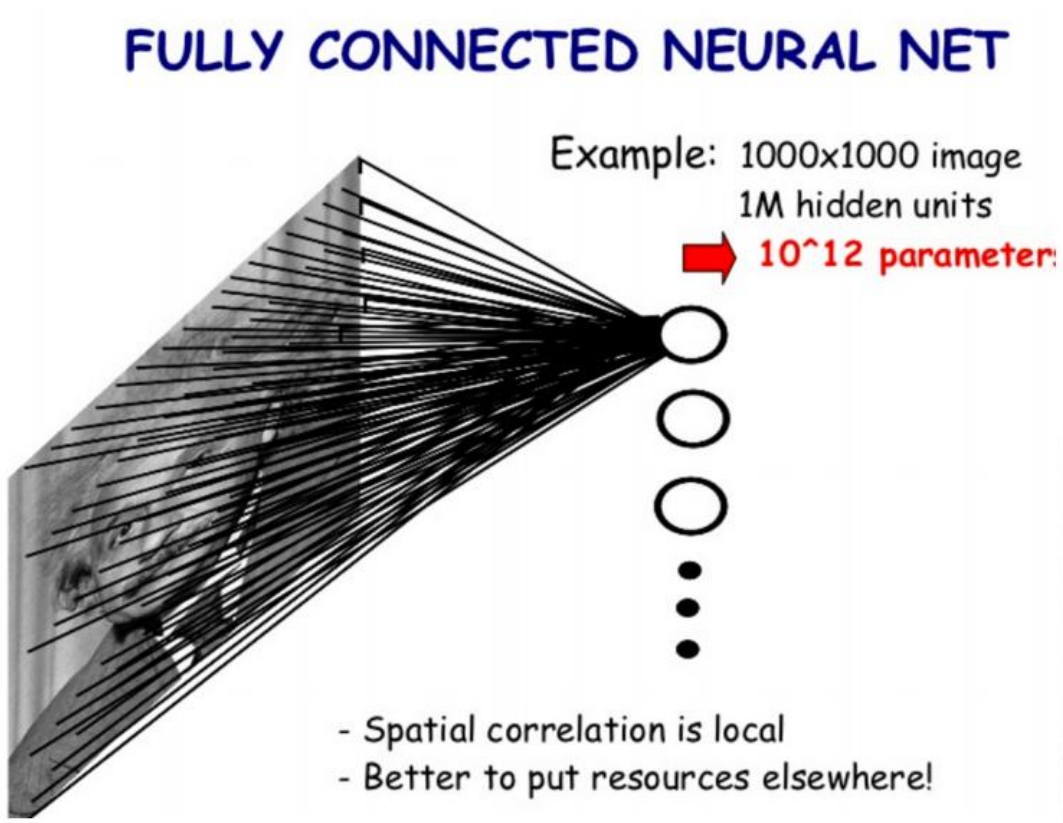
- Another common problem is that **MLPs react differently to an input (images) and its shifted version** — they are not translation invariant. Thus, spatial information is lost when the image is flattened (matrix to vector) into an MLP.

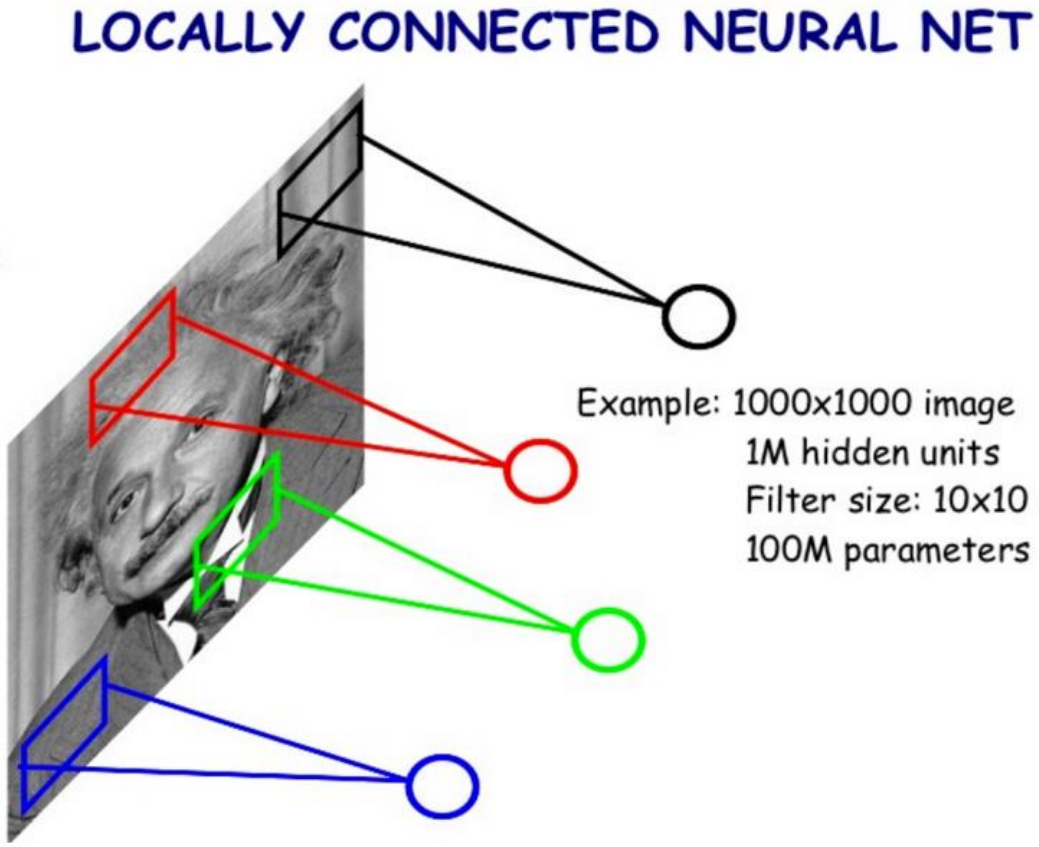
- For example, if a dog appears in the top left of the image in one picture and the bottom right of another picture, the MLP will try to correct itself and assume that a dog will always appear in this section of the image.



2. ARCHITECTURE

2.1. ADVANTAGES OF CNNs OVER MLPs





2. ARCHITECTURE

2.2. HOW A CNN LEARNS

1
Predict
The image passes through the network, which outputs a probability per class: dog 0.30, cat 0.70.

2
Measure the error
A loss function compares the prediction with the true label (dog) and returns one number: how wrong it was.

3
Backpropagate
The network works out how much each filter weight contributed to that error.

4
Update
Every weight moves a small step in the direction that reduces the error (gradient descent).

Repeat for every batch of images, over many epochs.

- **Nobody programs the filters.** They start as random numbers, and training turns them into edge, texture and shape detectors.
- This is why CNNs need **many labelled examples**: thousands of images per class is common.

2. ARCHITECTURE

2.3. KEY TERMS

Term	Meaning
Pixel	One number (0 to 255) per color channel at one position of the image
Channel	One layer of the image: 1 for grayscale, 3 for color (red, green, blue)
Kernel / filter	A small grid of weights (e.g. 3×3) that detects one pattern
Feature map	The output of one filter: shows where its pattern appears in the image
Stride / padding	How far the filter moves at each step / zeros added around the border
Softmax	The last step: turns scores into class probabilities that sum to 1
Epoch	One full pass over all the training images

AGENDA

1. Introduction
2. Architecture
3. **Building Blocks**
4. Hyperparameters & Training
5. Other considerations
6. Applications

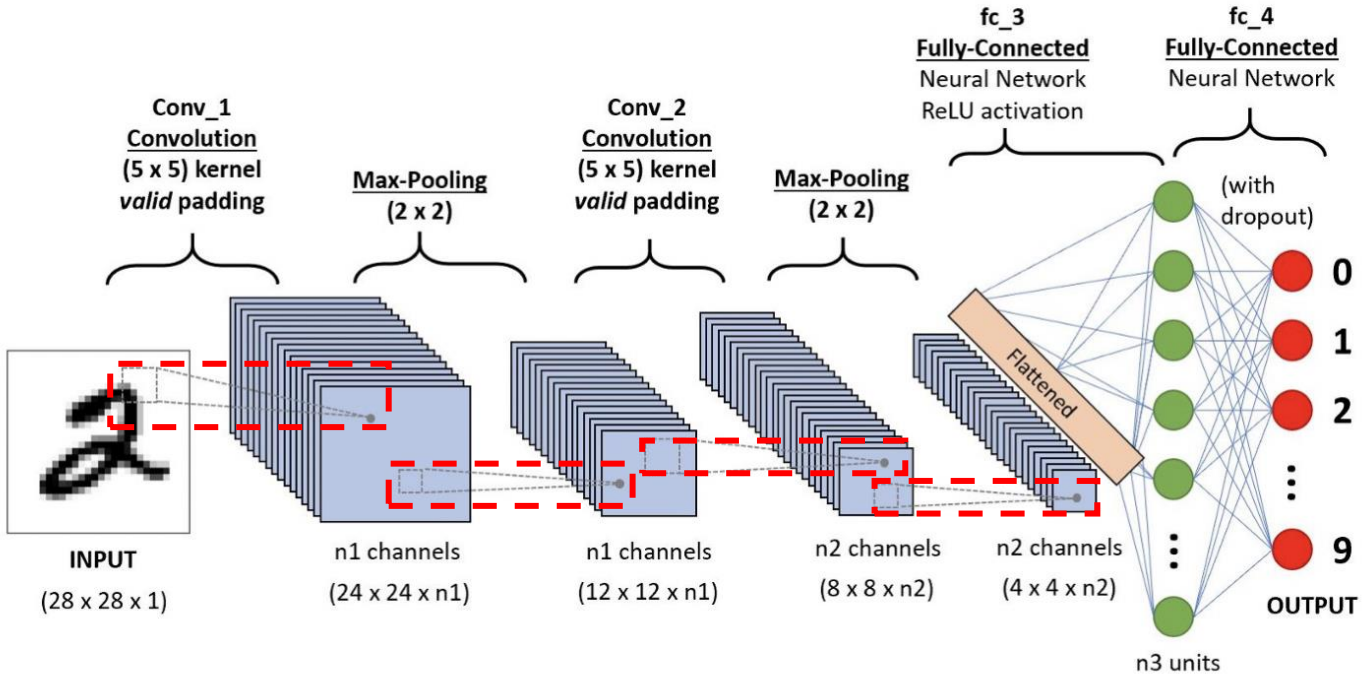


3. BUILDING BLOCKS

3.1. KERNEL

- In CNNs the vectors of weights and biases are called ***filters or kernels*** and represent features of the input (e.g., a particular shape).

- A distinguishing feature of CNNs is that many neurons can share the same filter. This reduces the memory footprint because a single bias and a single vector of weights are used across all receptive fields that share that filter, as opposed to each receptive field having its own bias and vector weighting



3. BUILDING BLOCKS

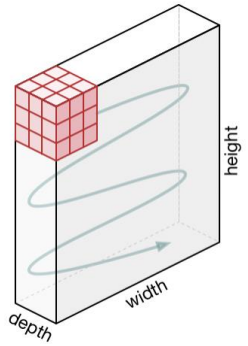
3.2. CONVOLUTION

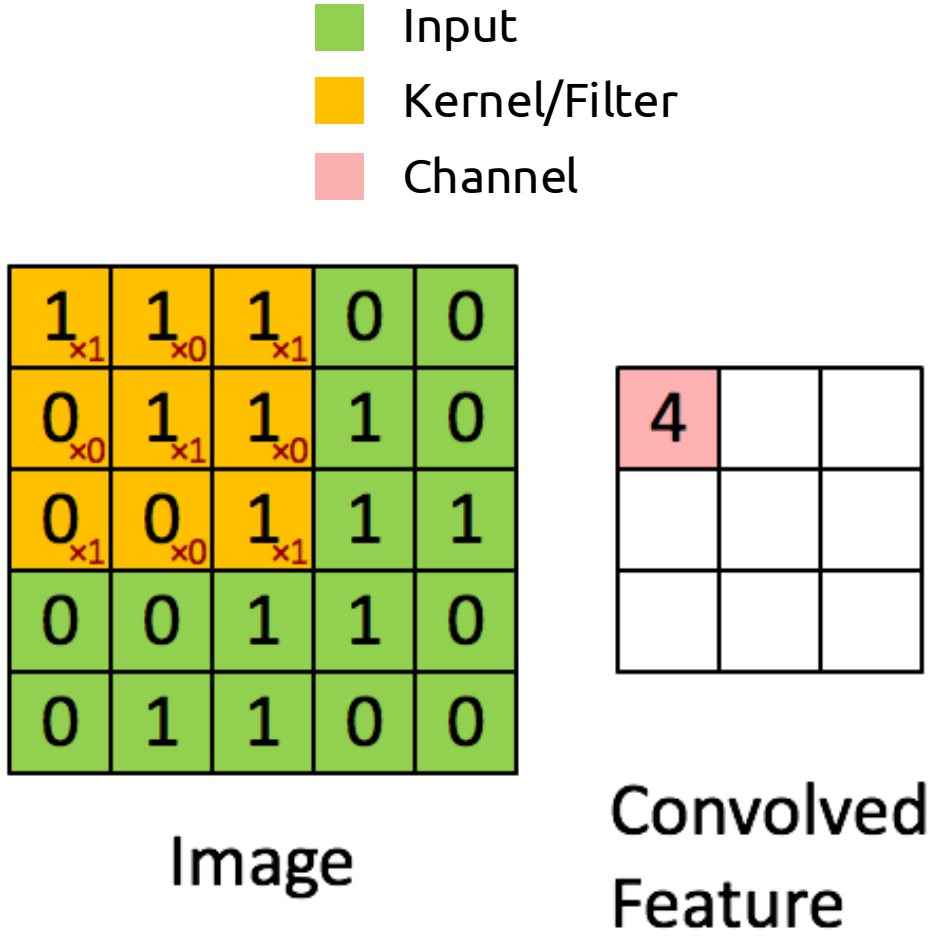
- The element involved in carrying out the convolution operation in the first part of a Convolutional Layer is called the ***kernel/filter***

- The Kernel:

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

- The Kernel shifts 9 times because of **Stride** is 1 (Non-Strided), every time performing a matrix multiplication operation between *K* and the portion *P* of the image over which the kernel is hovering.

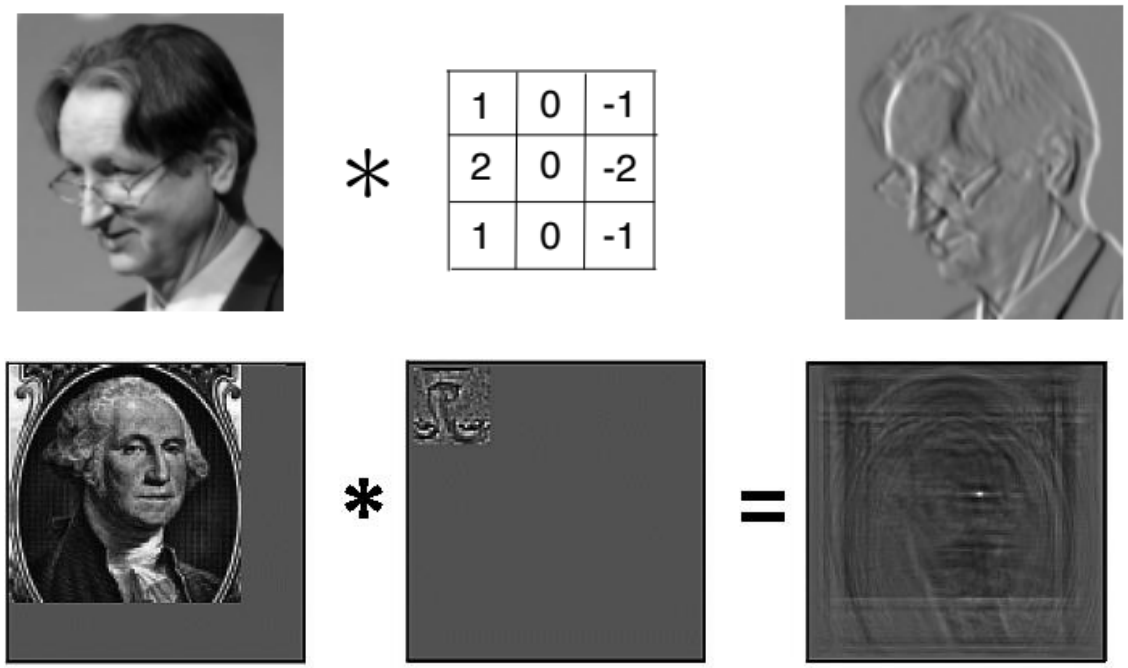


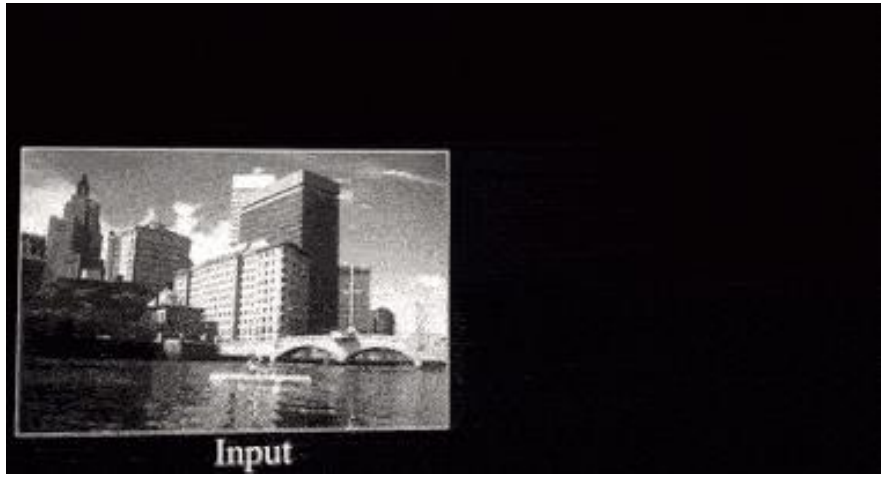


3. BUILDING BLOCKS

3.2. CONVOLUTION

- 2D convolutions are used as image filters, and when you would like to find a specific patch on an image:

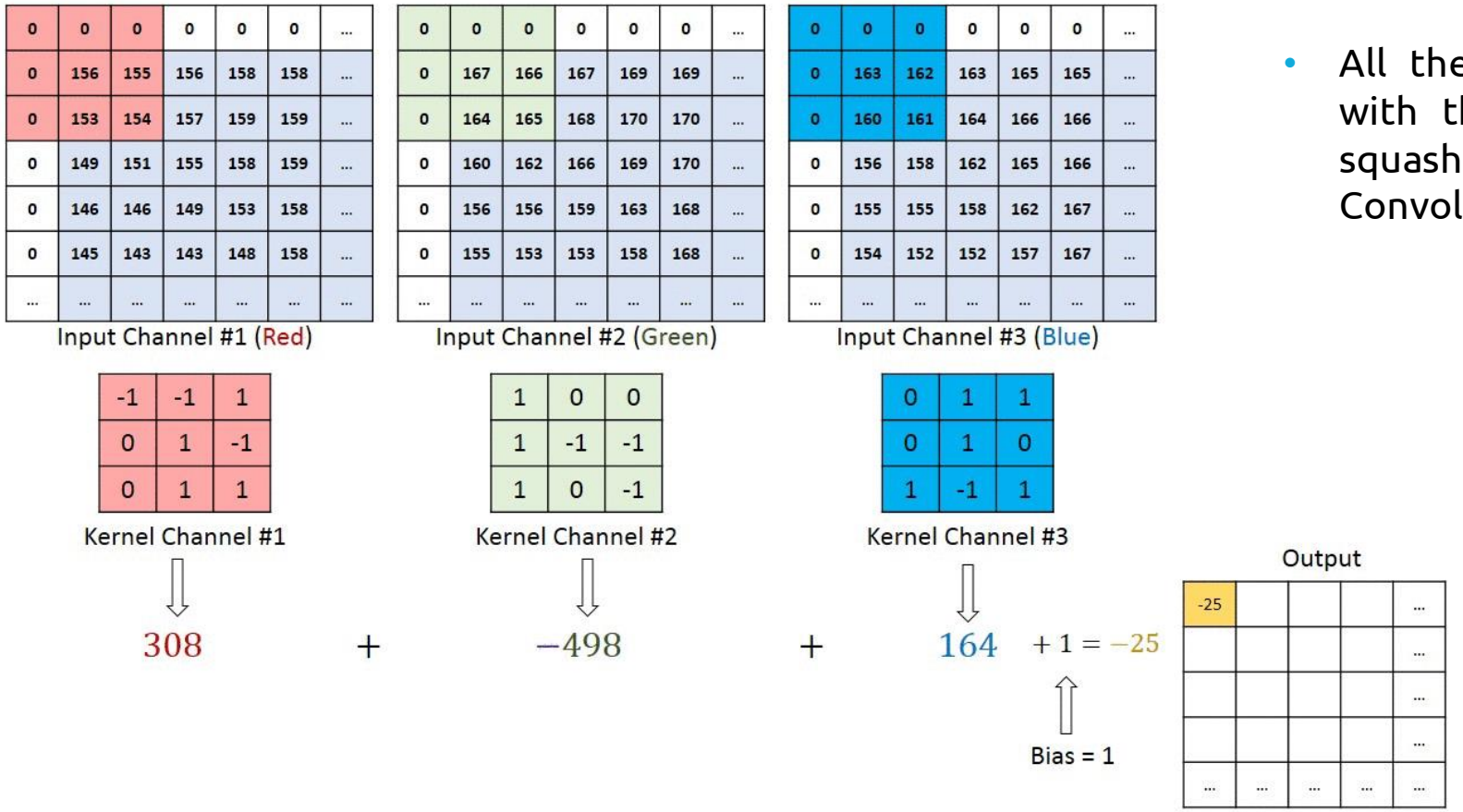




3. BUILDING BLOCKS

3.2. CONVOLUTION

- In the case of images with multiple channels (RGB), the kernel has the same depth as that of the input image.

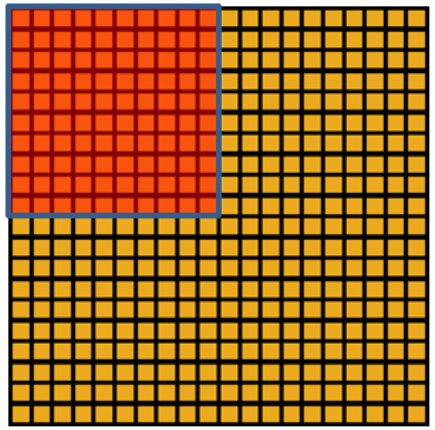


- All the results are summed with the bias to give us a squashed one-depth channel Convolved Feature Output.

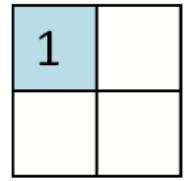
3. BUILDING BLOCKS

3.3. STRIDE

- By default, when we're doing convolution, we move our window one pixel at a time (stride=1), but sometimes in convolutional neural networks we want to move more than one pixel.
- We define stride as the step of the filter.
- For example, on pooling layers with kernels of size 2 we will use a stride of 2. Setting the stride and kernel size both to 2 will result in the output being exactly half the size of the input along both dimensions.



Convolved feature



Pooled feature

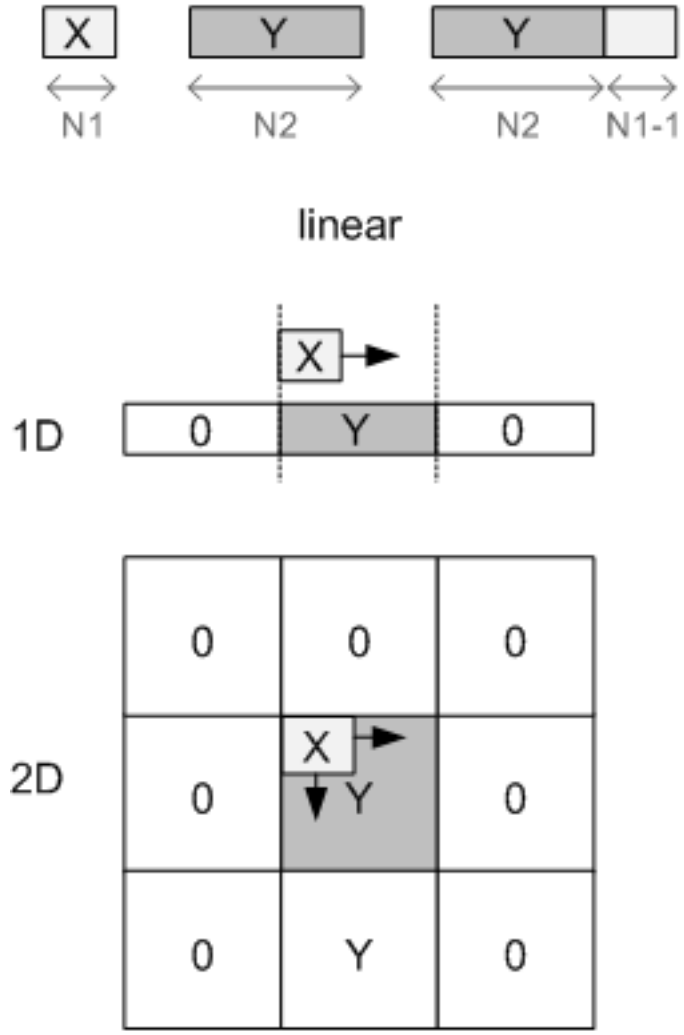
3. BUILDING BLOCKS

3.4. PADDING

- Padding adds a border of zeros around the input. Without it, every convolution shrinks the image: a 3×3 filter turns a 32×32 image into 30×30.

- Two common settings:
  - "valid" padding (P = 0): no zeros added, the output shrinks
  - "same" padding (P = 1 for a 3×3 filter): the output keeps the input size

- By using padding in a convolutional layer, we increase the contribution of pixels at the corners and the edges to the learning procedure.



3. BUILDING BLOCKS

3.5. OUTPUT SIZE

- It is useful to know what the dimensions of our output are going to be after we have performed some convolution operation. Luckily, there is a handy formula that tells us exactly that.

- If we consider convolving an input, of spatial **size** [*H*, *W*] **padded** by *P*, with a square *kernel* of size *F* and using **stride** *S*, then the output size of convolution is defined as:

$$outputSize_W = (W - F + 2P)/S + 1$$
$$outputSize_H = (H - F + 2P)/S + 1$$

EXAMPLE

Which is the output size if we convolve a 5x5x3 (WxHx3) input, with a conv layer with the following parameters?

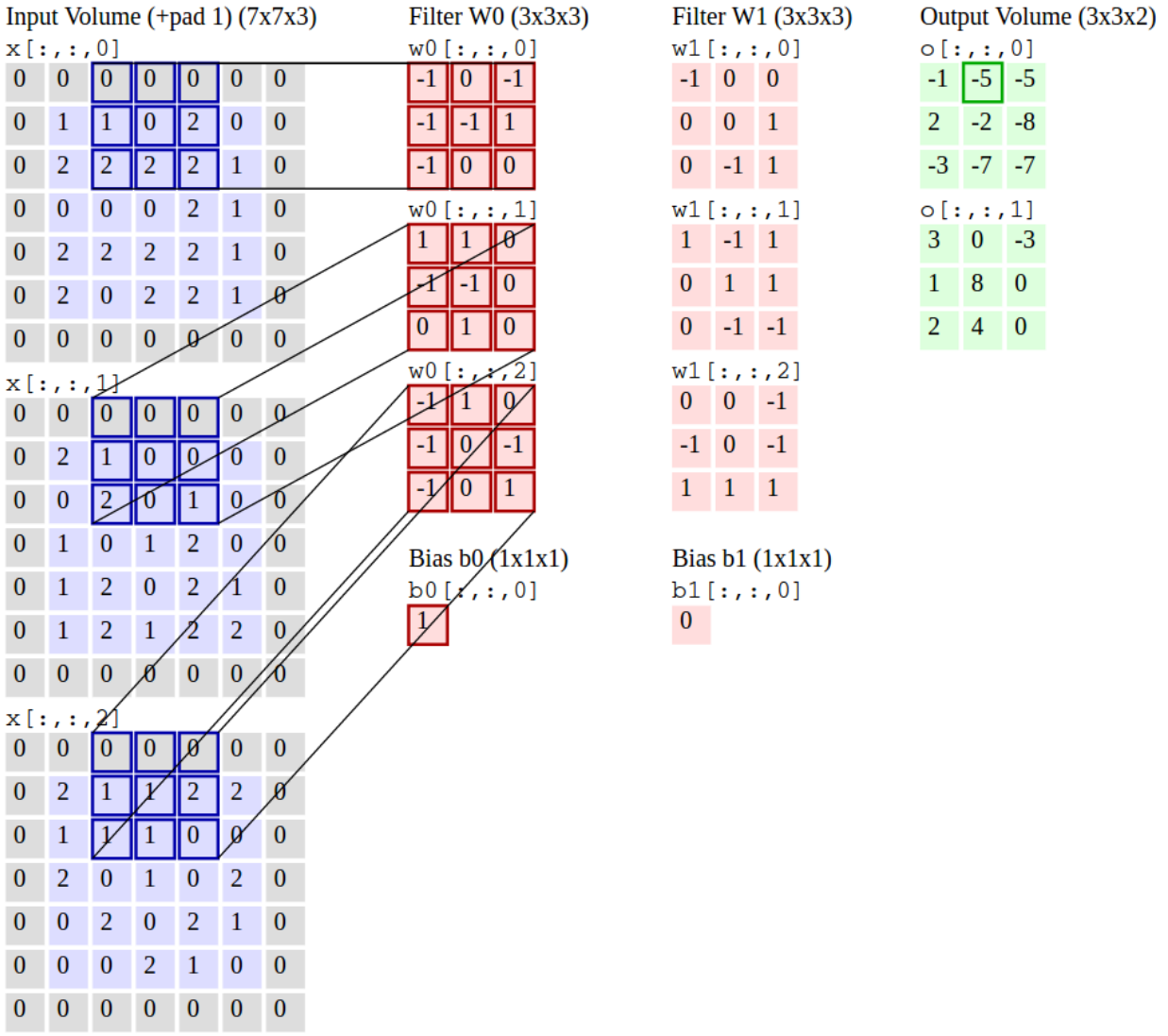
- Stride=2
- Pad=1
- F=3 (3x3 kernel)
- K=2 (two filters)

3. BUILDING BLOCKS

3.5. OUTPUT SIZE

- Our input has 3 channels, so we need a 3x3x3 kernel weight. We have 2 filters (K=2) so we will have 2 output activations at the end. Also, we can calculate the size of these two outputs to be: (5 - 3 + 2)/2 + 1 = 3.

- Looking at this example in more detail, basically we need to calculate 2 convolutions, one for each 3x3x3 filter and remembering to add the bias.

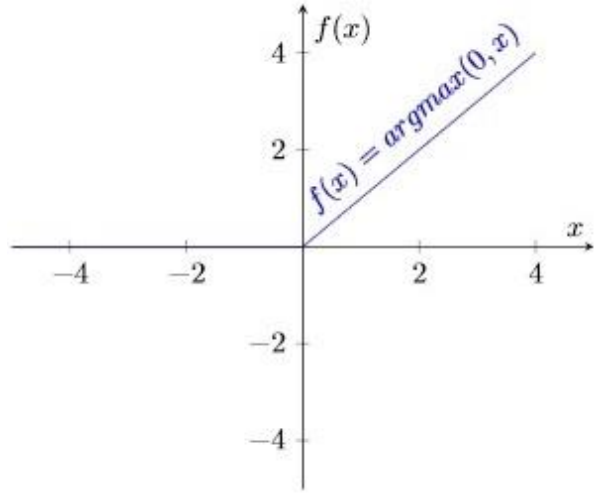


3. BUILDING BLOCKS

3.6. ACTIVATION FUNCTION: ReLU

- **ReLU (Rectified Linear Unit): f(x) = max(0, x).** Negative values become 0; positive values pass through unchanged.
- It is usually applied **after each convolution**, often after batch normalization. Without a non-linear function, a stack of layers behaves like one single linear layer and cannot learn complex shapes.
- Intuition: each value in a feature map says how strongly a pattern is present. ReLU **keeps the positive evidence** and drops the negative responses.
- ReLU is cheap to compute and lets deep networks **train faster** than older functions such as sigmoid or tanh.
- Not every layer uses it: **the output layer uses softmax** instead, and modern networks often use variants such as **Leaky ReLU or SiLU**.

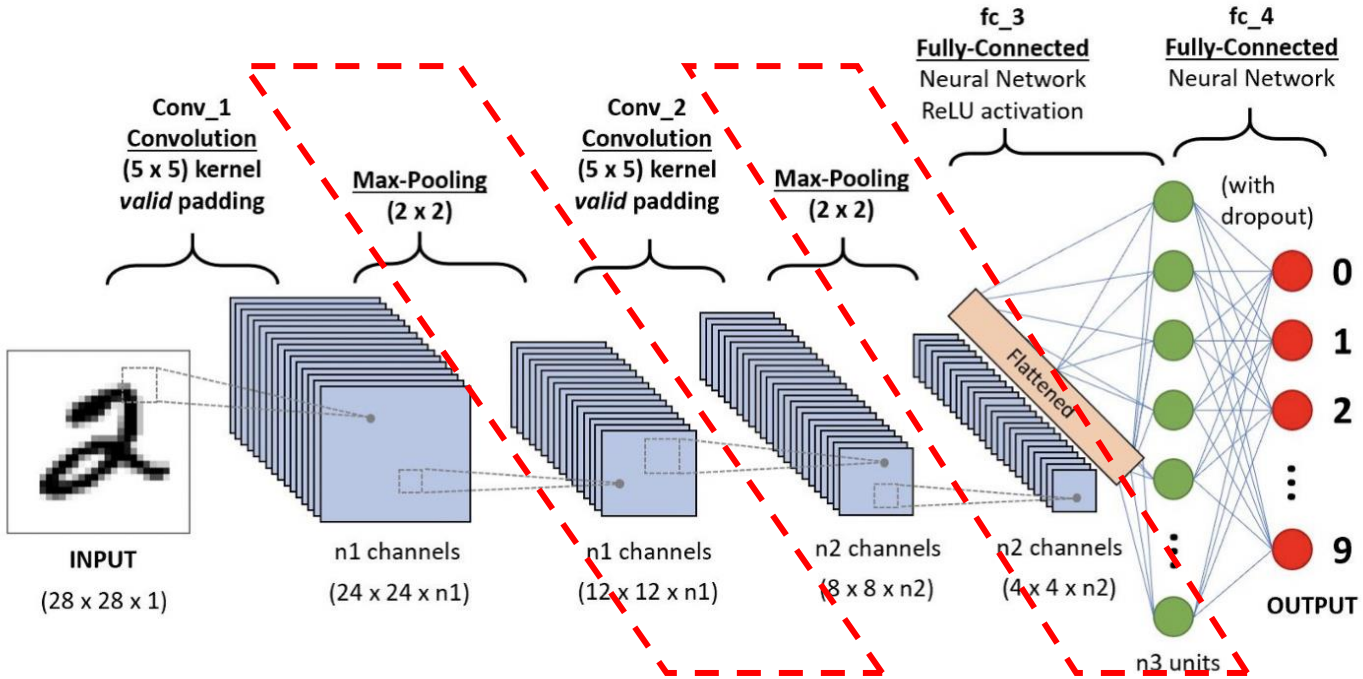
Rectified Linear Unit (ReLU)



3. BUILDING BLOCKS

3.7. POOLING LAYER

- **Pooling layers** reduce the dimensions of data by combining the outputs of neuron clusters at one layer into a single neuron in the next layer.
- Local pooling combines small clusters, tiling sizes such as 2 × 2 are commonly used. Global pooling acts on all the neurons of the feature map.
- There are two common types of pooling in popular use: **max and average**. Max pooling uses the maximum value of each local cluster of neurons in the feature map, while average pooling takes the average value.



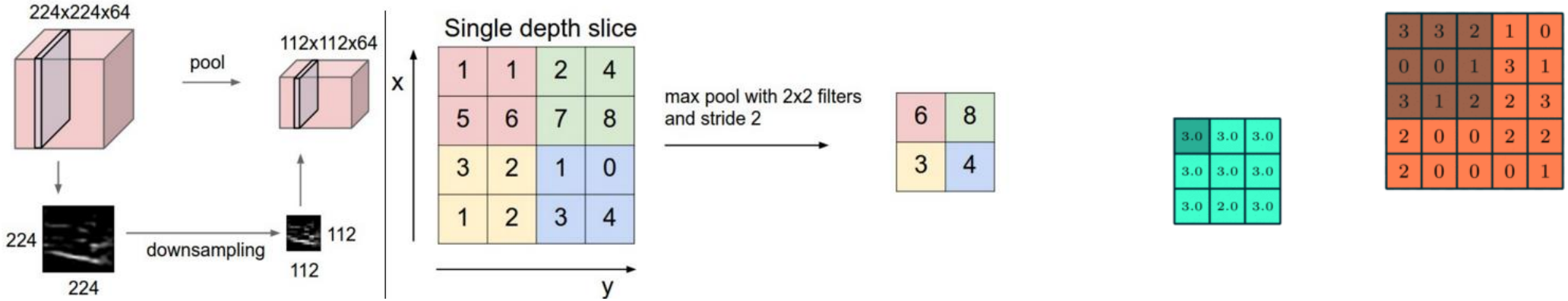
3. BUILDING BLOCKS

3.7. POOLING LAYER

- The pooling layer is used to reduce the spatial dimensions, but not depth, on a convolution neural network. Basically, this is what you gain:
  1. By having less spatial information you gain computation performance
  2. Fewer parameters in the next layers (pooling itself has no weights), so less chance to overfit
  3. Extracting dominant features

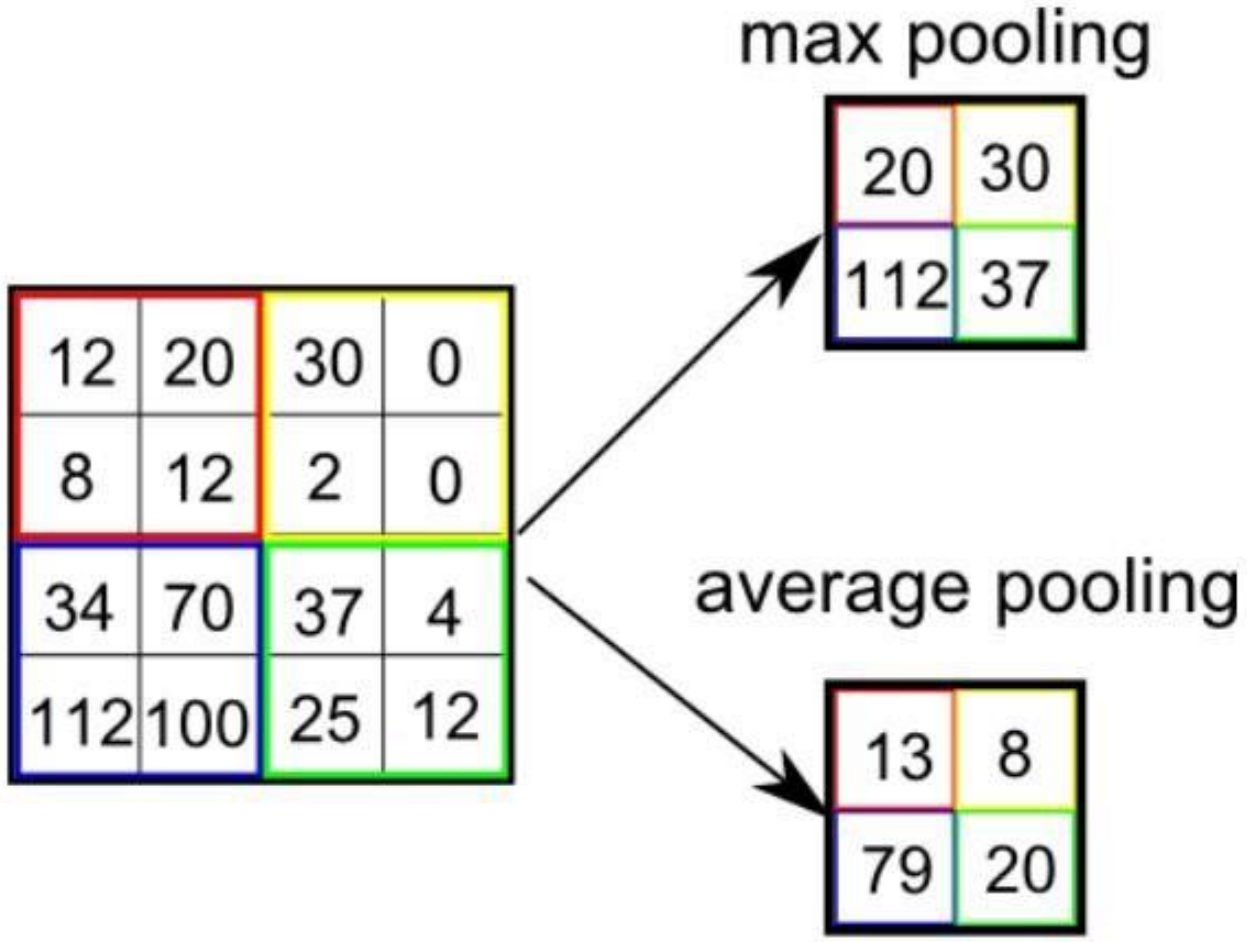
- Some projects don't use pooling, especially when they want to "learn" some object specific position. Learn how to play *Atari* games.

- Max-pooling layer:



3. BUILDING BLOCKS

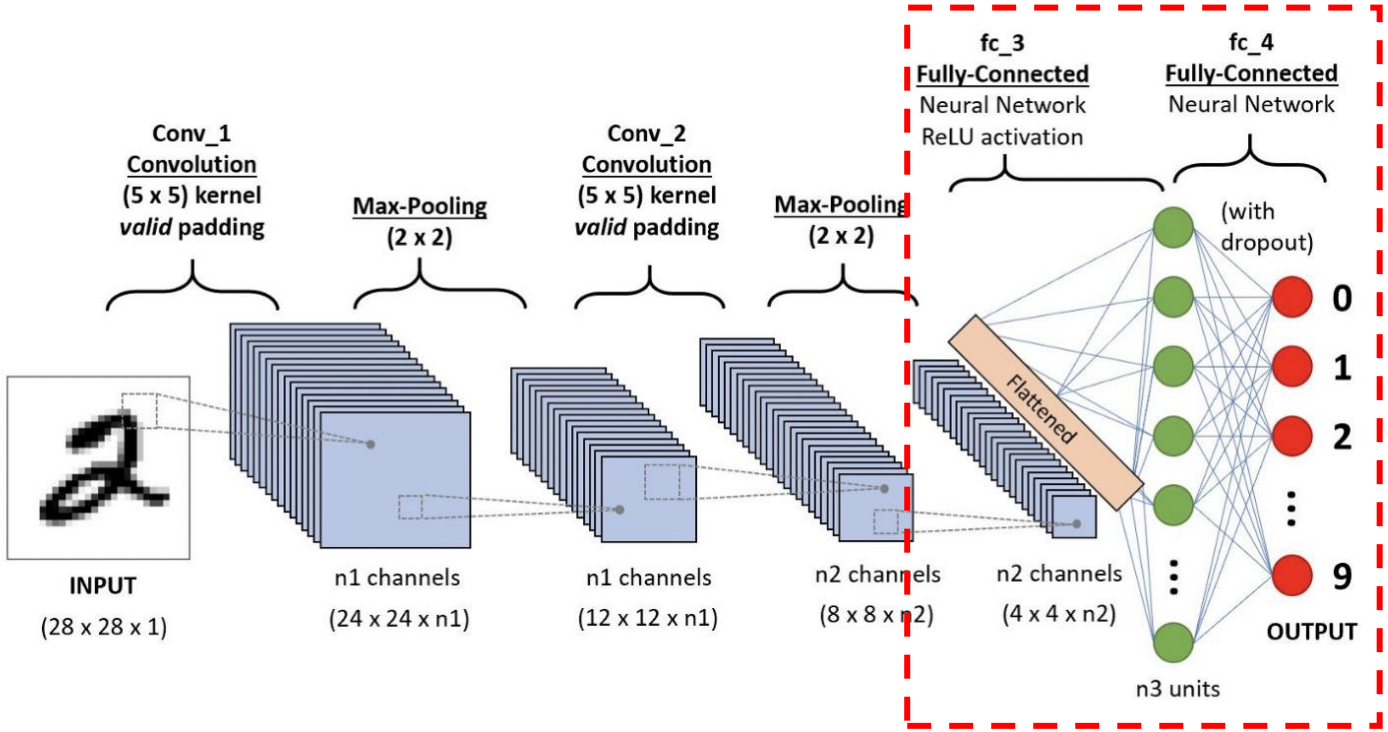
3.7. POOLING LAYER



3. BUILDING BLOCKS

3.8. FULLY CONNECTED LAYER

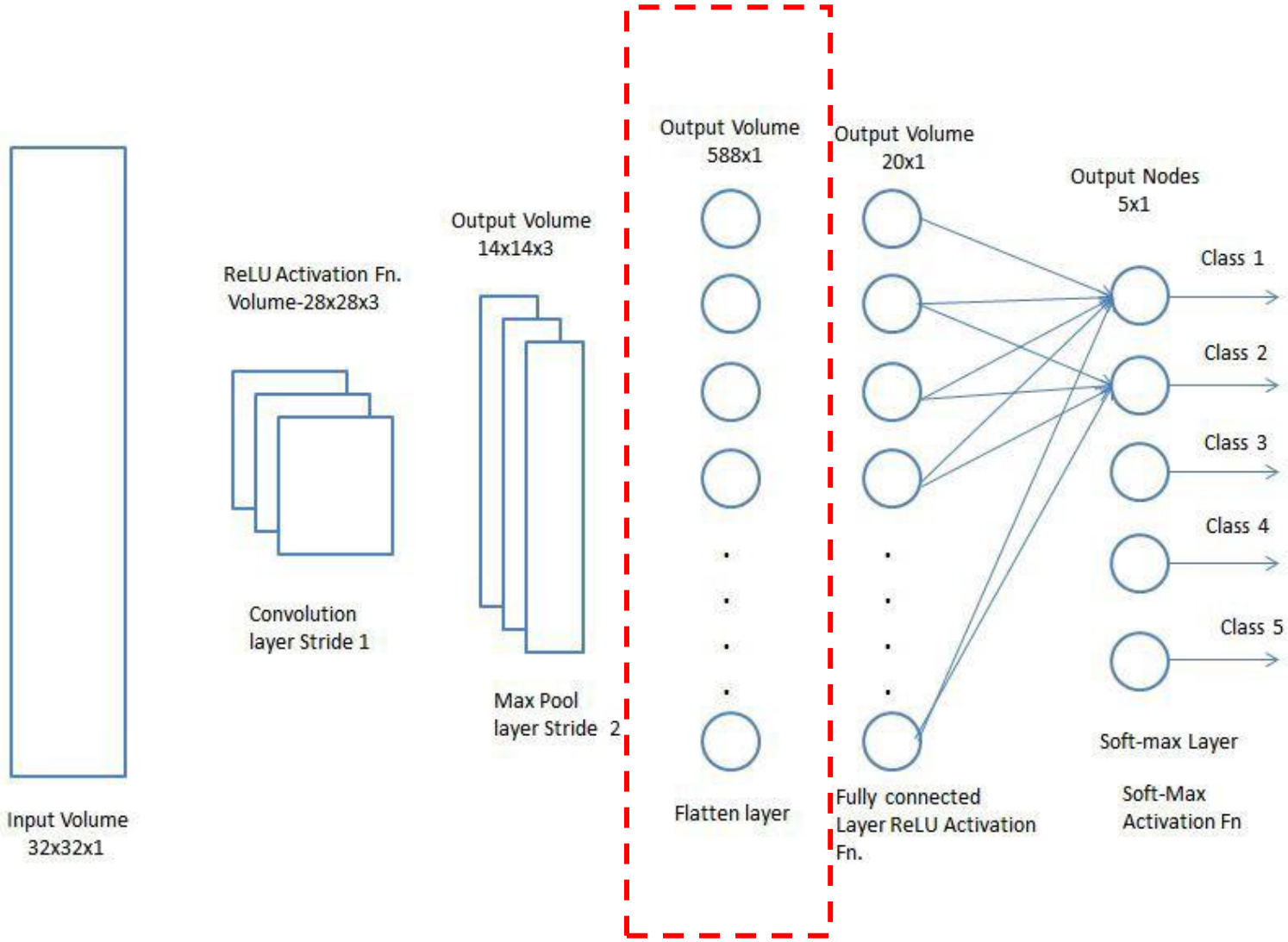
- Fully connected layers connect every neuron in one layer to every neuron in another layer. It is the same as a traditional multilayer perceptron neural network (MLP).
- The flattened matrix goes through a fully connected layer to classify the images.



3. BUILDING BLOCKS

3.8. FULLY CONNECTED LAYER

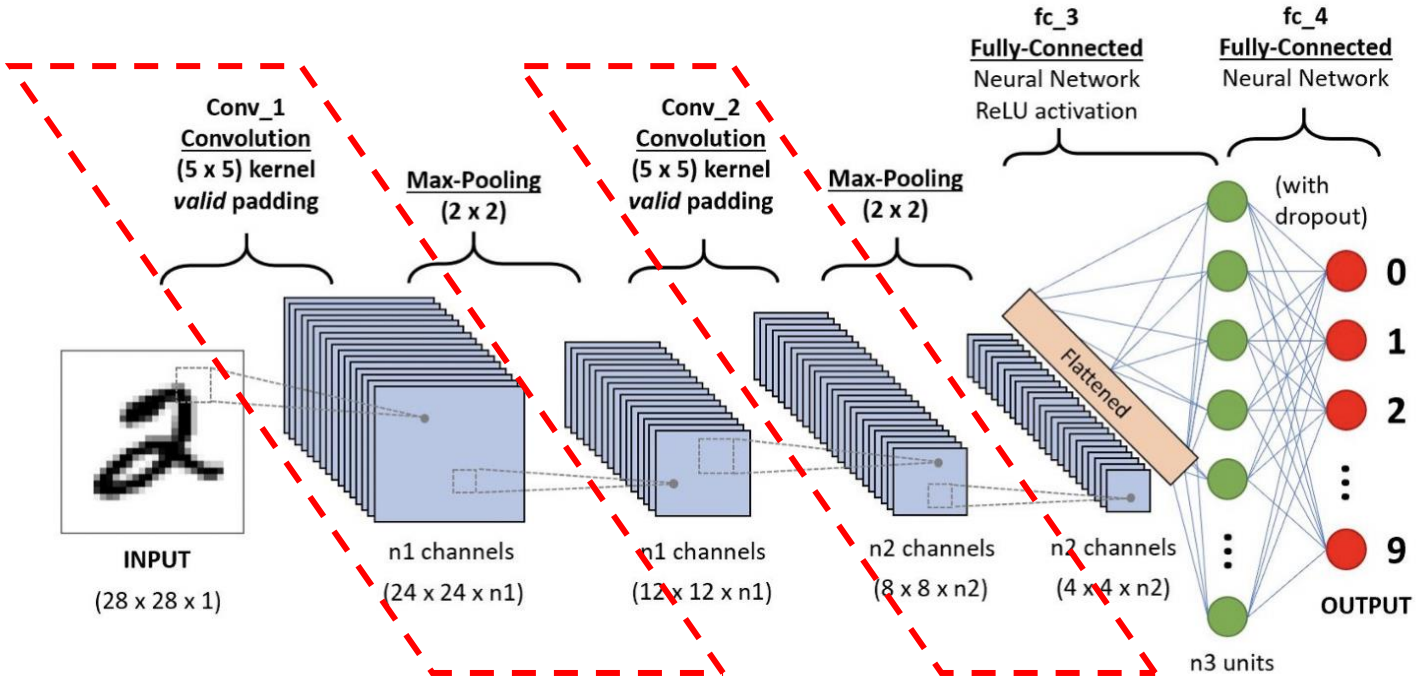
- The last feature maps are **flattened** into one vector.
- Fully connected layers **combine the high-level features** (eyes, ears, wheels) to decide the class.
- They often hold **most of the parameters**: about 90% of VGG-16's 138M.
- The last layer applies **softmax**, which turns scores into probabilities that sum to 1 (dog 0.92, cat 0.06, car 0.02).



3. BUILDING BLOCKS

3.9. PUTTING IT ALL TOGETHER

- A typical CNN repeats **convolution + ReLU → pooling**, then ends with **fully connected** layers and **softmax**.
- Early layers detect edges; deeper layers detect parts and whole objects.



Input:

(number of inputs) × (input height) × (input width) × (input channels)

Output:

(number of inputs) × (feature map height) × (feature map width) × (feature map channels)

AGENDA

1. Introduction
2. Architecture
3. Building Blocks
4. Hyperparameters & Training
5. Other considerations
6. Applications



4. HYPERPARAMETERS & TRAINING

4.1. DESIGN CHOICES AND HYPERPARAMETERS

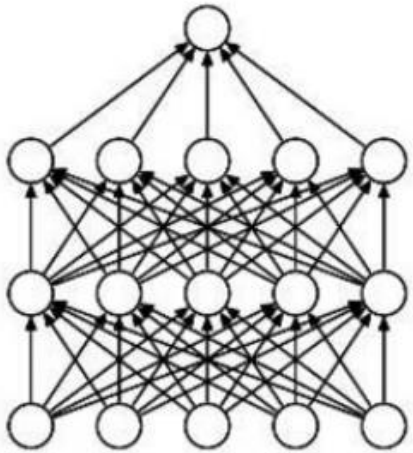
Kernel Size	The number of pixels processed together. The challenge is to find the right level of granularity so as to create abstractions at the proper scale, given a particular data set, and without overfitting.
Number of Filters	Since feature map size decreases with depth, layers near the input layer tend to have fewer filters while higher layers can have more.
Pooling Type	Max-pooling vs Avg-pooling vs Others
Training settings	Learning rate (size of each weight update), batch size (images per update) and epochs (full passes over the training data). Unlike the weights, these are set before training.

4. HYPERPARAMETERS & TRAINING

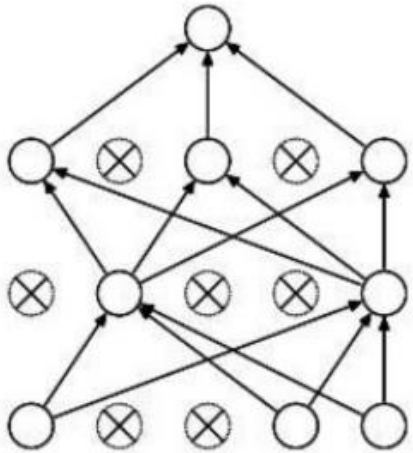
4.2. OVERFITTING AND DROPOUT

Dropout

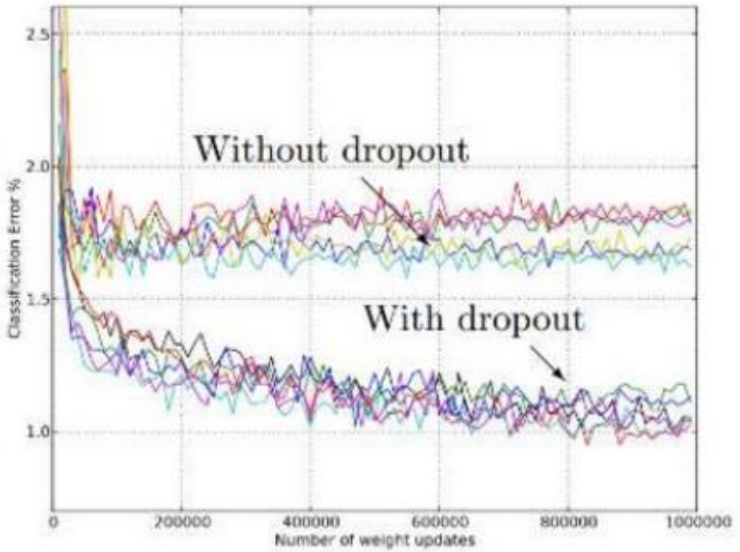
- **Overfitting:** the network memorizes the training images and fails on new ones, e.g. 99% accuracy on training data but 70% on test data.
- **Dropout** randomly switches off a share of neurons (often 50%) at each training step, so the network cannot rely on any single neuron and learns more general features. It is turned off at prediction time.



(a) Standard Neural Net



(b) After applying dropout.



4. HYPERPARAMETERS & TRAINING

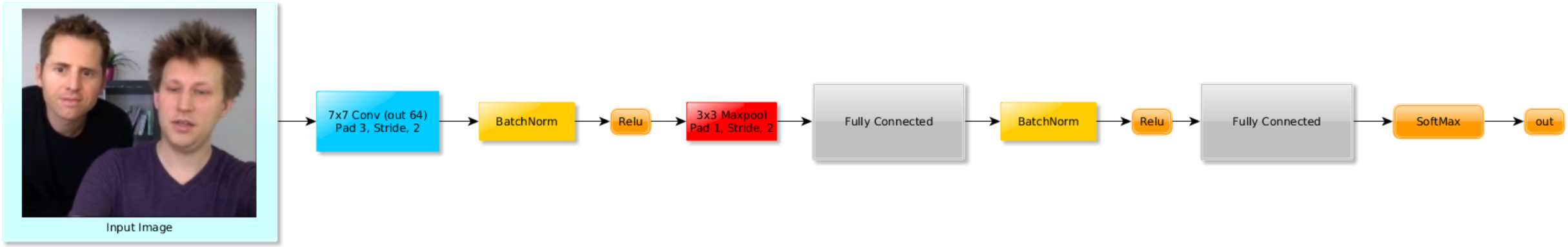
4.3. BATCH NORMALIZATION

- **Problem:** as the weights change during training, the values passed between layers keep changing scale, which slows learning down.
- **Batch normalization** rescales a layer's outputs, for each mini-batch, to mean 0 and standard deviation 1. The network then learns its own scale and shift for each channel.
- What you gain:
    1. Faster, more stable training, even for very deep networks such as ResNet
    2. Higher learning rates can be used
    3. Less sensitivity to how the weights are initialized
    4. A mild regularization effect, often used together with dropout

4. HYPERPARAMETERS & TRAINING

4.3. BATCH NORMALIZATION

- Batch normalization is placed **after a convolution or fully connected layer and before ReLU**: Conv → BatchNorm → ReLU.
- During training it uses the statistics of the current batch; at prediction time it uses averages stored during training.



4. HYPERPARAMETERS & TRAINING

4.4. TRANSFER LEARNING AND DATA AUGMENTATION

Transfer learning

- Start from a network **already trained on ImageNet** (e.g. ResNet, EfficientNet) instead of random weights.
- Its early layers already detect edges and textures; **only the last layers are retrained** for the new classes.
- Works with a few hundred images per class and trains in minutes instead of days.

Data augmentation

- Create new training images by **transforming the ones you have**: flips, rotations, crops, zooms, brightness changes.
- The label stays the same: a flipped dog is still a dog.
- The network sees more variety, so it **overfits less** and copes better with new photos.

AGENDA

1. Introduction
2. Architecture
3. Building Blocks
4. Hyperparameters & Training
5. Other considerations
6. Applications



5. OTHER CONSIDERATIONS

5.1. TYPES OF COMPUTER VISION TASKS

Classification

"What is in the image?"

Output: One label for the whole image: "dog, 92%"

Networks: ResNet, EfficientNet

Example: Sorting product photos by category

Object detection

"Which objects are there, and where?"

Output: A box and a label for each object

Networks: YOLO, Faster R-CNN

Example: Counting cars at a crossing

Segmentation

"Which pixels belong to which object?"

Output: A label for every pixel

Networks: U-Net, Mask R-CNN

Example: Outlining a tumor in an MRI scan

Detection and segmentation networks use the same building blocks; only the output layers change.

5. OTHER CONSIDERATIONS

5.2. FAMOUS NETWORKS AND LIBRARIES

Network	Year	Why it matters
LeNet-5	1998	First practical CNN: read handwritten digits on bank cheques
AlexNet	2012	Won ImageNet and started the deep learning boom
VGG-16	2014	Simple, deep design built from stacks of 3×3 filters
GoogLeNet	2014	Inception modules: several filter sizes in parallel
ResNet	2015	Skip connections make networks with 100+ layers trainable
U-Net	2015	Pixel-level segmentation, widely used in medical imaging
YOLO	2016	Real-time object detection in a single pass
MobileNet	2017	Light enough to run on phones and edge devices
EfficientNet	2019	Scales depth, width and image resolution together

Libraries used today: PyTorch, TensorFlow / Keras, JAX.

5. OTHER CONSIDERATIONS

5.3. LIMITATIONS AND VISION TRANSFORMERS

- **Need large labelled datasets**: labelling medical or industrial images is slow and expensive.
- **Hard to explain**: it is difficult to say why the network decided something (tools such as Grad-CAM help).
- **Easy to fool**: tiny pixel changes or unusual lighting can change the prediction.
- **Local view**: each filter sees a small region, so relations between distant parts of the image are learned slowly.

	CNN	**Vision Transformer (ViT)**
How it reads the image	Small local patches, combined layer by layer	Splits the image into patches and relates all of them at once (attention)
Data needed	Works well with small and medium datasets	Needs very large datasets or a pre-trained model
Where it is used	Phones, edge devices, medical imaging	Large-scale models; hybrids such as ConvNeXt mix both ideas

AGENDA

1. Introduction
2. Architecture
3. Building Blocks
4. Hyperparameters & Training
5. Other considerations
6. Applications



6. APPLICATIONS

6.1. IMAGE RECOGNITION

- **2012:** AlexNet won the ImageNet challenge (1.2M images, 1,000 classes) with 15% top-5 error, against 26% for the next team.
- **2015:** ResNet reached 3.6% top-5 error, below the roughly 5% measured for humans on the same task.
- Beyond classifying a whole image, CNNs **detect** objects with boxes (YOLO) and **segment** every pixel (U-Net).
- Everyday uses: photo search by content, face unlock on phones, product recognition in shops, screening of X-rays and skin lesions.

6. APPLICATIONS

6.2. VIDEO ANALYSIS

- A video is a **sequence of images**, so it adds a time dimension.
- Two common approaches: **3D convolutions** that slide over space and time at once, or a **2D CNN on each frame** followed by a model that combines the frames.
- Uses: action recognition in sports footage, traffic and crowd monitoring, and driver-assistance cameras that track pedestrians and lanes.

6. APPLICATIONS

6.3. NATURAL LANGUAGE PROCESSING

- Text can be treated as a **1D sequence**: a filter slides over groups of 3 to 5 words and detects phrases.
- Around 2014 to 2017, CNNs gave strong results in sentence classification and sentiment analysis (Kim, 2014).
- Since 2018, **Transformers** (BERT, GPT) have replaced CNNs for most language tasks. CNNs remain useful for small, fast text classifiers.

6. APPLICATIONS

6.4. OTHER APPLICATIONS



Medical imaging

Spot tumors and fractures in X-ray, CT and MRI scans



Quality inspection

Find scratches and defects on production lines



Driver assistance

Detect pedestrians, lanes and traffic signs



Agriculture

Diagnose crop disease from photos of leaves



Anomaly detection

Flag unusual patterns in images or sensor data



Drug discovery

Predict properties of molecules and proteins



Games

AlphaGo used CNNs to evaluate Go board positions



Audio and time series

Classify sounds from spectrograms and forecast signals

6. APPLICATIONS

6.5. DEMO: CHEST CT CLASSIFICATION IN TENSORFLOW

PROBLEM:

Classify chest CT scans into 4 classes: three types of lung cancer or normal tissue.

Chest CT-Scan images Dataset

SOLUTION:

A small CNN trained with TensorFlow / Keras reaches 90% accuracy on the test images.

Chest CT scan Classification(90% test acc)

STEPS

1. Load the images and resize them to 224×224
2. Augment: random flips, rotations and zooms
3. Train the CNN and watch validation accuracy
4. Evaluate once on the held-out test set

```
model = keras.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(4, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=20)
```

KEY TAKEAWAYS

WHAT TO REMEMBER FROM THIS SESSION

1

An image is a **grid of numbers**; a CNN learns which patterns in that grid matter.

2

A **filter** slides over the image and produces a **feature map** showing where its pattern appears.

3

Stride and padding set the output size: (W − F + 2P) / S + 1.

4

ReLU adds non-linearity, **pooling** shrinks the maps, **fully connected + softmax** classify.

5

The filters are **learned with backpropagation** from labelled examples, not designed by hand.

6

Dropout, batch norm, augmentation and transfer learning fight overfitting and small datasets.

REFERENCES

FURTHER READING

- Hubel, D. H. & Wiesel, T. N. (1962). Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. Journal of Physiology.
- Fukushima, K. (1980). Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition. Biological Cybernetics.
- LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE.
- Krizhevsky, A., Sutskever, I. & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. NeurIPS.
- Srivastava, N. et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. JMLR.
- Kim, Y. (2014). Convolutional neural networks for sentence classification. EMNLP.
- Ioffe, S. & Szegedy, C. (2015). Batch normalization: accelerating deep network training. ICML.
- Ronneberger, O., Fischer, P. & Brox, T. (2015). U-Net: convolutional networks for biomedical image segmentation. MICCAI.
- He, K., Zhang, X., Ren, S. & Sun, J. (2016). Deep residual learning for image recognition. CVPR.
- Redmon, J. et al. (2016). You only look once: unified, real-time object detection. CVPR.
- Dosovitskiy, A. et al. (2021). An image is worth 16x16 words: transformers for image recognition at scale. ICLR.
- Stanford CS231n: Convolutional Neural Networks for Visual Recognition, course notes (cs231n.github.io).


THANK YOU